

CSE 202 Midterm 1 Part (b)

Olcay Taner YILDIZ

I. MULTIPLE CHOICE QUESTIONS (40 POINTS)

- Which of the following functions has the largest growth?
 - n^2
 - n
 - $n \log(n)$
 - 1.01^n
 - n^{10}
- What is the time complexity of the following fragment of code?


```
for (int i = 1; i <= N; i *= 2)
    sum += i * i;
```

 - $\mathcal{O}(N)$
 - $\mathcal{O}(1)$
 - $\mathcal{O}(N^2)$
 - $\mathcal{O}(\log N)$
 - $\mathcal{O}(\sqrt{N})$
- What is the time complexity of the following fragment of code?


```
int i = 1;
while (i < N) {
    i = i + 2;
}
```

 - $\mathcal{O}(N)$
 - $\mathcal{O}(1)$
 - $\mathcal{O}(N^2)$
 - $\mathcal{O}(\sqrt{N})$
 - $\mathcal{O}(\log N)$
- What is the value of *sum* after the following code fragment is executed?


```
int sum = 0;
for (i = 0; i < N * N; i++)
    for (j = 0; j < N * N; j++)
        sum++;
```

 - $N^2 + N^3$
 - N^3
 - N^2
 - N
 - N^4
- Given the time complexity equation for a recursive algorithm as $T(N) = 2T(N/2) + \mathcal{O}(N)$. What is the time complexity of that algorithm?
 - $\mathcal{O}(N^2 \log N)$
 - $\mathcal{O}(N^2)$
 - $\mathcal{O}(N^3)$
 - $\mathcal{O}(N \log N)$
 - $\mathcal{O}(N)$
- Given the time complexity equation for a recursive algorithm as $T(N) = 8T(N/2) + \mathcal{O}(N^2)$. What is the time complexity of that algorithm?
 - $\mathcal{O}(N^2 \log N)$
 - $\mathcal{O}(N^2)$
 - $\mathcal{O}(N^3)$
 - $\mathcal{O}(N \log N)$
 - $\mathcal{O}(N)$
- If you do not use a last pointer in the doubly linked list implementation, what will be the time complexity of *insertFirst*?
 - $\mathcal{O}(1)$
 - $\mathcal{O}(N)$
 - $\mathcal{O}(\log N)$
 - $\mathcal{O}(N^2)$
 - $\mathcal{O}(N \log N)$
- If you do not use a last pointer in the doubly linked list implementation, what will be the time complexity of *insertLast*?
 - $\mathcal{O}(1)$
 - $\mathcal{O}(N)$
 - $\mathcal{O}(\log N)$
 - $\mathcal{O}(N^2)$
 - $\mathcal{O}(N \log N)$
- Given the following data *a* = (4-3-8-9-2) as a singly linked list. What will be the linked list after the following operations?


```
a.deleteLast
a.insertLast(3)
a.deleteFirst()
a.insertFirst(4)
a.insertFirst(9)
```

 - 9-4-3-8-9
 - 9-4-8-9-3-4
 - 9-3-4-8-9-3
 - 9-4-3-8-9-3
 - 4-3-8-9-3-9
- What is the time complexity of the *insertLast* for singly linked list, doubly linked list, and circular linked list?
 - $\mathcal{O}(1)$, $\mathcal{O}(1)$, $\mathcal{O}(1)$
 - $\mathcal{O}(N)$, $\mathcal{O}(N)$, $\mathcal{O}(N)$
 - $\mathcal{O}(N)$, $\mathcal{O}(1)$, $\mathcal{O}(1)$
 - $\mathcal{O}(N)$, $\mathcal{O}(1)$, $\mathcal{O}(N)$
 - $\mathcal{O}(1)$, $\mathcal{O}(1)$, $\mathcal{O}(N)$

11. Given that there are more than two nodes in a single linked list, which of the following code fragment **deletes** the **second** node?

- a) `first.next.next = first.next;`
- b) `first.next = first.next.next;`
- c) `first = first.next;`
- d) `first.next = first;`
- e) `first = first.next;`
`first.next = first.next.next;`

12. Given that there are more than one nodes in a single linked list, which of the following code fragment **inserts** `newNode1` and `newNode2` one after another at the **end** of the list?

- a) `last.next = newNode1;`
`last.next = newNode2;`
- b) `last.next = newNode1;`
`last.next = newNode2;`
`last = newNode2;`
- c) `last.next = newNode1;`
`last = newNode1;`
`last.next = newNode2;`
`last = newNode2;`
- d) `last = newNode1;`
`last = newNode2;`
- e) `last = newNode2;`
`last.next = newNode1;`
`last.next = newNode2;`

13. Given that there are more than one node in a doubly linked list, which of the following code fragment **deletes** the **first** node?

- a) `first = first.next;`
- b) `first = first.next;`
`first.previous = null;`
- c) `first.previous = null;`
`first = first.next;`
- d) `first.previous = first;`
- e) `first = first.previous;`
`first.next = first;`

14. Which of the following code fragment **removes** a node from the **middle** of a doubly linked list, where **previous** shows the node before the deleted node?

- a) `previous.next = previous.next.next;`
`previous.next.next.previous = previous;`
- b) `previous.next.next.previous = previous;`
`previous.next = previous.next.next;`
- c) `previous = previous.next;`
`previous.next = previous.next.next;`
- d) `previous.next = previous;`
`previous.next = previous.next.next;`
- e) `previous.next.previous = previous;`
`previous.next = previous.next.next;`

15. Which of the following orderings can be obtained for 1,2,3,4 if those numbers must be pushed in that order but can be popped in any order?

- a) 4-1-2-3
- b) 4-1-3-2

- c) 3-4-1-2
- d) 3-4-2-1
- e) 4-2-3-1

16. What are the time complexities of push and pop operations for array implementation?

- a) $\mathcal{O}(1)$, $\mathcal{O}(N)$
- b) $\mathcal{O}(N)$, $\mathcal{O}(N)$
- c) $\mathcal{O}(N)$, $\mathcal{O}(1)$
- d) $\mathcal{O}(\log N)$, $\mathcal{O}(1)$
- e) $\mathcal{O}(1)$, $\mathcal{O}(1)$

17. Given the following data `a = (4-3-8-9-2)` as a stack, where 2 is the at the top of the stack. What will be the stack after the following operations?

```
a.push(3)
a.pop
a.pop
a.pop
a.push(4)
```

- a) 4-8-9-2
- b) 4-3-8-2
- c) 4-3-8-4
- d) 3-8-9-2
- e) 4-4-9-2

18. Given the stack is not empty, which of the following code fragment pops an element from the stack for the linked list implementation? (Does not return)

- a) `top = top.next.next;`
- b) `top.next = top;`
- c) `top = top.next;`
- d) `top.next.next = top;`
- e) `top.next = top.next.next;`

19. Given that the stack is not empty, which of the following code fragment pops an element from the stack for the array implementation? (Does not return)

- a) `top++;`
- b) `top--;`
- c) `top += 2;`
- d) `top -= 2;`
- e) None of the above

20. If there are N elements in the stack, how many pop operations can be done one after another at most before there is stack underflow?

- a) $N + 2$
- b) $N + 1$
- c) N
- d) $N - 1$
- e) $N - 2$